SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

DAY 10

Functional and Non-Functional Requirements

Student Book • Module II: Requirements Engineering

Learning focus
Identify what a system must do, the quality levels it must achieve, and how to express both types of requirements so they can be understood and tested.

Running case study: College Event Management System (CEMS)

Designed for classroom projection, discussion, practical application, and revision.

1. Learning Outcomes

By the end of this session, you should be able to:

• Explain the difference between functional and non-functional requirements.

• Classify requirements from a real software scenario.

• Recognise common quality attributes such as performance, security, usability, reliability, and availability.

• Rewrite vague statements as clear, verifiable requirements.

• Create a small requirement catalogue for the College Event Management System.

> **Core idea:** functional requirements describe the services and behaviours a system provides; non-functional requirements describe qualities, constraints, or conditions under which those services must operate.

2. Understanding Functional Requirements

A **functional requirement (FR)** states a service, task, response, or behaviour that the system must provide. It often describes an action performed by a user or by the system in response to an event.

Examples for CEMS

• The system shall allow students to view published college events.

• The system shall allow an authenticated student to register for an event with available capacity.

• The system shall prevent registration when an event reaches its capacity.

• The system shall allow an authorised coordinator to create and update event details.

• The system shall send a registration confirmation after a successful registration, if email notification is in scope.

A functional requirement often answers

• Who interacts with the system?

• What action can they perform?

• What information is used or produced?

• What rules or conditions apply?

• What should happen when the action succeeds or fails?

> **Example:** “The system shall allow a coordinator to publish an approved event.” This identifies an actor (coordinator), an action (publish), an object (event), and a condition (approved).

3. Understanding Non-Functional Requirements

A **non-functional requirement (NFR)** describes a quality attribute, operating condition, or constraint. NFRs influence the user experience and the technical architecture even when they do not introduce a new user-facing feature.

Common categories

Category	What it concerns	Example requirement
Performance	Response time, throughput, resource use	The event list shall load within two seconds for 95% of requests under the agreed normal-load test.
Security	Authentication, authorisation, confidentiality, integrity	Only users with the coordinator role shall create or publish events.
Usability / accessibility	Ease of learning, clarity, accessibility	Registration errors shall identify the field that needs correction in clear language.
Reliability	Correct operation over time; failure frequency	A failed registration shall not create a confirmed registration record.

Category	What it concerns	Example requirement
Availability	When the service is accessible	The service shall meet an agreed monthly availability target, excluding defined maintenance windows.
Scalability	Handling growth in users or data	The system shall support the agreed peak number of concurrent users without breaching performance targets.
Maintainability	Ease of diagnosing and changing the system	The registration rules shall be covered by automated tests and documented.
Compatibility / portability	Supported devices, browsers, platforms	The student registration flow shall work on the agreed supported desktop and mobile browsers.
Privacy / compliance	Handling of personal data and policy obligations	Access to student contact details shall be restricted to authorised roles.

4. Functional vs Non-Functional: Side-by-Side

Question	Functional requirement	Non-functional requirement
Focus	What the system does	How well, under what conditions, or within what constraints it operates
Typical wording	The system shall allow / calculate / display / save / prevent…	The system shall respond within / support / restrict / remain available…
Example	Students can register for events.	Registration confirmation shall be displayed within two seconds under agreed load.
How to test	Check the behaviour and its outcomes.	Measure or inspect a quality attribute against a defined criterion.
Why it matters	Delivers the required service.	Makes the service usable, dependable, secure, and fit for its context.

Some requirements overlap in practice. For example, “only coordinators can publish events” is a system behaviour, but it also expresses a security rule. Classify it according to the purpose of the requirement and keep its security implications visible.

5. The Problem With Vague Requirements

A requirement is not useful merely because it sounds positive. Words such as “fast,” “easy,” “robust,” “secure,” and “user-friendly” need a shared meaning or a way to verify them.

Vague statement	Improved, verifiable statement
The system should be fast.	The event list shall load within two seconds for 95% of requests under the agreed normal-load test.
The app must be secure.	Only authenticated users shall access personal registration details; access attempts by unauthorised users shall be denied and logged.
The system should support many users.	The system shall support 500 concurrent users while meeting the specified response-time target in the agreed load test.
The registration process should be easy.	In usability testing, at least 90% of representative first-time users shall complete registration without assistance.
The system should always work.	The service shall meet an agreed availability target, with measurement period and planned maintenance defined.

> These sample values are illustrative, not universal standards. A real project must agree on measurable targets based on business needs, risk, budget, and expected usage.

6. How to Write a Strong Requirement

Use wording that is specific, singular where practical, feasible, necessary, consistent, and verifiable. Give each requirement a unique identifier so that it can be tracked.

A simple writing pattern

ID + system + required behaviour or quality + condition + measurable acceptance criterion (when relevant)

Examples

• FR-01: The system shall display all published events to an authenticated student.

• FR-02: The system shall reject a registration request when the event has no remaining capacity.

• NFR-PERF-01: Under the agreed normal-load test, 95% of event-list requests shall complete within two seconds.

• NFR-SEC-01: A student-role account shall not be permitted to create or publish an event.

Checklist for reviewing a requirement

• Clear: Would different readers interpret it in the same way?

• Necessary: Does it support a stakeholder need, business goal, policy, or constraint?

• Feasible: Can the team realistically meet it?

• Consistent: Does it conflict with another requirement?

• Verifiable: Can evidence show whether it has been satisfied?

• Traceable: Can it be linked to its source and related tests?

• Prioritised: Is its importance known?

7. Requirement Categories Beyond FR and NFR

Projects may use more detailed classifications to make requirements easier to organise:

Category	Description	CEMS example
Business requirement	High-level outcome the organisation wants.	Reduce duplicate registrations and improve event coordination.
User requirement	A goal or task expressed from a user's perspective.	As a student, I want to find upcoming events so I can choose which to attend.
Functional requirement	A system behaviour or service.	The system shall allow students to register for an event.
Non-functional requirement	A quality attribute or operating constraint.	The event list shall meet the agreed response-time target.
Interface requirement	How the system interacts with users or other systems.	The registration page shall display event name, date, venue, capacity status, and registration action.
Data requirement	Rules about data content, quality, retention, or relationships.	Each registration shall reference one student and one event.
Constraint	A boundary the solution must respect.	The system must use the college's approved identity provider, if mandated.

8. Case Study: Build a Requirement Catalogue

Scenario: CEMS helps students discover college events and register online. Coordinators create events and track capacity. The college wants reliable access and protection of student data.

ID	Type	Requirement	Priority
FR-01	Functional	The system shall display published events to students.	Must
FR-02	Functional	The system shall allow a student to register for an event with available capacity.	Must
FR-03	Functional	The system shall reject registration when event capacity is reached.	Must
FR-04	Functional	The system shall allow authorised coordinators to create and update events.	Must
NFR-01	Performance	95% of event-list requests shall complete within two seconds under the agreed normal-load test.	Should / agree target
NFR-02	Security	Only authorised coordinator accounts shall publish events.	Must
NFR-03	Usability	Error messages shall identify the problem and explain the next corrective action.	Should
NFR-04	Reliability	A failed registration shall not be recorded as confirmed.	Must

Notice that priorities are project decisions, not properties that come automatically from the requirement type. A performance target may be critical for one system and less critical for another.

9. Turning Requirements Into Acceptance Tests

Requirements should guide testing from the beginning. A test needs a condition, an action, and an observable expected result.

Requirement	Test idea	Expected result
FR-02: Register when capacity is available.	Given an event with available seats, submit a valid student registration.	Registration is saved and confirmation is shown.
FR-03: Reject when full.	Given an event at capacity, attempt to register another student.	Registration is rejected and a clear message is shown.
NFR-01: Event list performance.	Run the agreed load test and measure response times.	At least 95% of requests complete within the agreed two-second target.
NFR-02: Restrict publishing.	Sign in as a student and attempt to publish an event.	Action is denied; no event is published.

A requirement that cannot be verified may be too vague, may lack an agreed measurement, or may need to be rewritten. Test design can therefore reveal requirement problems early.

10. Common Mistakes

• Writing a feature list and forgetting performance, security, reliability, accessibility, and privacy.

• Confusing a solution choice with the actual need. For example, “use a blue button” may be a design choice rather than a stakeholder requirement.

• Using absolute words such as “always” or “never” without defining realistic conditions.

• Combining several behaviours into one requirement, making it hard to test and trace.

• Inventing numerical targets without stakeholder agreement or evidence.

• Assuming every quality requirement has equal priority for every project.

11. Classroom Activity

Activity: Classify, improve, and test

In pairs, classify each statement as functional, non-functional, constraint, or ambiguous. Rewrite vague statements and suggest one test or verification method.

Statement	Your task
The system shall let students cancel a registration before the published deadline.	Identify the behaviour and a useful boundary test.
The website must be very fast.	Rewrite with a measurable response-time target and test condition.
Only administrators may assign coordinator roles.	Classify the rule and describe an unauthorised-access test.
The system shall support 300 concurrent users.	Identify the quality attribute and define how to test it.
The system must use the college-approved sign-in service.	Identify the constraint and what evidence could verify it.

12. Knowledge Check

1. What is the main difference between FRs and NFRs?

FRs describe system behaviours or services; NFRs describe quality attributes, operating conditions, or constraints.

2. Is “The system shall prevent a student from publishing an event” functional or non-functional?

It states observable system behaviour; it can be documented as a functional access-control requirement, while also supporting the security quality attribute.

3. Why is “the system should be easy to use” weak?

“Easy” is subjective unless the expected users, task, and success measure are defined.

4. Give one performance NFR for CEMS.

Example: 95% of event-list requests complete within two seconds under an agreed normal-load test.

5. Why should each requirement have an ID?

To track its source, priority, status, related design, tests, and changes.

6. Can a requirement be functional and security-related at the same time?

Yes. Classification helps organise it; it does not remove cross-cutting quality implications.

7. What makes a requirement verifiable?

There is a practical way to inspect, measure, test, or otherwise obtain evidence that it is satisfied.

8. Should a team invent performance targets without stakeholder agreement?

No. Targets should be agreed using business needs, risk, expected usage, and feasibility.

13. Glossary

Term	Meaning
Functional requirement	A required system service, behaviour, or response.
Non-functional requirement	A required quality, operating condition, or constraint.
Performance	How quickly or efficiently a system responds or processes work.
Availability	The extent to which a service is accessible when needed.
Reliability	The ability to operate correctly and consistently over time.
Usability	How effectively and efficiently intended users can use the system.
Acceptance criterion	A condition or piece of evidence used to determine whether a requirement is satisfied.
Traceability	The ability to follow a requirement to its source and related project artefacts.

14. Homework and Deliverables

Use CEMS or another approved system, such as online food ordering, hospital appointment booking, library management, or student attendance.

• Write at least six functional requirements and five non-functional requirements.

• Give every requirement a unique ID and a priority.

• Rewrite three vague statements into verifiable requirements.

• For four requirements, write an acceptance test with precondition, action, and expected result.

• Identify two constraints or policies that may apply to your system.

• Create a small table linking requirement IDs to their stakeholder source and test case.

Suggested submission structure

1. System overview
2. Requirement catalogue
3. Functional / non-functional classification
4. Improved vague requirements
5. Acceptance tests
6. Constraints and assumptions

Next session: Requirement elicitation techniques and stakeholder analysis in greater depth, with interview planning and practical examples.